Bash Script For Linux

1. **Types Of Linux :**
   - **Root User**
   - **Regular User**
   - **Service User**

i. **Root user** is a main user account in linux systems. It is automatically created during the installation. It has the highest privilege in the system. It can do any administrative work and can access any service. It should not be used for routine activities . it can't be deleted. But if required, it can be disabled.

ii. **Regular User** is the normal user account. During the installation, one regular user account is created automatically. After the installation , we can create as many regular user accounts as we need. This account has moderate privilege. The account is intended for routine work. It can perform only the tasks for which it is allowed and can access only those files and services for which it is authorized. As per requirement, it can be deleted.

iii. **Service User** is created by installation packages when they are installed. These accounts are used by services to run processes and execute functions. These accounts are either intended nor should be used for routine work.

2. Basic linux administration commands :

- **hostname -** This command helps to print the hostname and helps to connect other systems.
- **uname -** This command helps to print the current operating system of your machine.
- **uname -a** - This command will print the more information of the current machine with the hostname and the version.
- **who** - This command helps to show the current user and their logged-in time.
- **whoami** - This command helps to know your name.
- **id** - After this command it will print the UID, GID, group id, cdrom, sudo,or etc..,
- **id root** - This command will be printed about root specification.
- **id "user"** - (put your user name) and this command will be printed about user specification.

3. Adding and Deleting a user :

- So, here I am going to **add a user**, by using this command "**sudo adduser user1**" (**root user** can only add a user.) and after this command it will be ask **some information** like password, name, address, phone number (if won't to give info then you may to give pass to continue)

- Delete the user by this command “**sudo userdel user1**” to delete the user.

4. **Adding and Deleting a user :**
   - **Adding** a group by using this command “**sudo groupadd developers**” will create a group to check the group by using this command “**cat /etc/group**”. If you want to add a specific user in a group, then enter this command “**sudo usermod -a -G developers user1**” now see the user will be added.
   - **Deleting the group** using this command “**sudo gpasswd -d user1 developers**”. Delete the group by using “**sudo groupdel developers**”

5. **Basic to Advance linux commands :**
   - **pwd** - prints out the working directory
   - **cd** - change directory
   - **ls** - list files in the directory
   - **ls -la** - list all hidden directory with permissions
   - **ls -l** - list all directory with permissions
   - **mkdir** - make a directory
   - **rmdir** - delete a directory
   - **touch** - create empty file
   - **rm** - deletes a file or files
   - **date +%H-%M-%S** - prints out the current date

- **cut** - cuts a specific part of text (**cat file.txt | cut -c 1**)
- **cp** - copy files from one file to another
- **mv** - moves files from one file to another
- **wc** - counts number of lines, words or characters in a file **(cat file.txt | wc -c), (cat file.txt | wc -l), (cat file.txt | wc -w)**
- **less** - shows only the text which fits on the screen
- **man** - gives a manual entry of a command
- **head** - prints out first 5 line of a file
- **tail** - prints out last 5 lines of a file
- **kill** - stops a specific type of task which is not responding
- **whereis** - finds out the file location **(whereis /home)**
- **whatis** - gives details about a file **(whatis bash)**
- **uptime** - prints out the uptime of the linux system
- **sudo** - run commands as administrator
- **history** - prints out the history of commands you have run
- **file** - gives information about a file **(file num.txt)**
- **sort** - sorts unique text from a file

6. **Hello World in bash**

- **TR command :**
- The tr command in unix is a command line utility for translating or deleting characters. It supports a range of transformations including uppercase to lowercase, squeezing repeating characters, deleting specific characters and basic find and replace. It can be used with UNIX pipes to support more complex translations. Tr stands for translate.
- **Example :**
- **cat wle.txt| tr "[a-z]" "[A-Z]"**
- **cat wle.txt| tr "[a-z]" "[A-Z]"**

- **Xargs command :**
- The xargs command will be run like if you were to have a big file and there lines are in decent form. But if you want to in one line then use xargs command like : **cat file.txt | xargs**

- **Echo options :**
- **-n** - do not print the trailing newline
- **-e** - enable interpretation of backslash escapes.
- **\b** - backspace (echo -e “hello **\b**world”)
- **** - backlash
- **\n** - new line (echo -e **-n** “hello **\n**world”)
- **\r** - carriage return

- **\c** - it will be remove after the words
- **\t** - horizontal tab (echo -e **"\t**welcome to **\tt**hello **\t**world")
- **\v** - vertical tab (echo -e "welcome **\v**to **\v**this **\v**world")
- **echo {1..10}** - it will print 1 to 10 number range
- Enter **nano hello_world.sh** and write **echo hello world** and save it. (echo is used to print the name which you gave.) and also you can enter **printf "%s" hello world !** (it will print on a command line) and if you enter this **print "%s\n" "hello world !"** (it will print on a new line).
- Give permission by using **chmod +x hello_world.sh**
- Run bash by using **./hello_world.sh**

7. She-Bang

- The she-bang is just a way to tell your terminal to use this bash and there are many kinds of shells.
- Format for she-bang **#!/bin/bash # - Sharp ! - Bang** → **Sharp bang** → **Sha bang**
- Before entering any code on your terminal you have to enter this she-bang for formalities. (#!/bin/bash)

8. Variables

- The variable is a value that can change, depending on conditions or on information passed to the program. A variable in bash can contain a number, a character, or a string of characters.
- **Rules:** variable name can't be started from a **number** it can only be started with an **underscore or alphabet**.
- And **spaces** are not allowed in a variable.
- **Three ways to assign a value to a variable.**
  1) **Explicit: VAR=value**
  2) **Read command: read VR**
  3) **Command substitution: VAR=$(ls)**
- **You can write this.**

```
#!/bin/bash

VAR=Deepak

echo $VAR
```

- Save and execute the script.

4. **Read command in bash**
   - The read input works like it will **store and print to you.**

```
#!/bin/bash
echo "what is your name: "
```

read name
echo your name is $name

- Save the script and run.
- read name - it will be take as a storage
- $name - it will print what you have been saved.

5. **Advance method to read user input**
   - #!/bin/bash
     read HOSTNAME < /etc/hostname
     echo Hostname $HOSTNAME

   - **read HOSTNAME < /etc/hostname** - this input will be taken from the hostname file.
   - **echo Hostname $HOSTNAME** - it will print what they are taken from the hostname file.
   - Save the file and run.

6. **Command substitution**
   - **#!/bin/bash**
     VAR=$(pwd)
     echo Your path is : $VAR
   - Save and run the script. It will print the current directory and you can change the value of **VAR.** like: whoami, pwd, dir, etc..,
   - There are different ways to use the command substitution.

7. **Builtin Or Environment variables in bash**
   - The environment variables are pre-defined in our system by entering **“env”.**
   - These are some env command - **echo $HOSTNAME**

     echo $CDPATH
     echo $COLUMNS
     echo $BASH_VERSION
     echo $ZSH_VERSION
     echo $BASHOPTS
     echo $HISTFILESIZE
     echo $HISTFILE
     echo $HISTSIZE
     echo $SECONDS
     echo $UID
     echo $DIRSTACK
     echo $MAILCHECK
     echo $PATH
     echo $PS1
     echo $PS2
     echo $PS4
     echo $SHELLOPTS

   - **Projects on variable :**

     In this script we are calculating time to execute a script.

#Starting time

- **START=‘date +%s‘**
  pwd
  sleep 2

  #Ending time
  END=’date +%s’
  TIME_TAKEN=$((END-START))

  #Time taken to execute the script
  Echo Time taken to execute the script : $TIME_TAKEN
- Save the file and give permission. And execute the script.

8. **Arithmetic Calculation In Bash Script :**
   - Using **let**
   - Using **(( ))**
   - Using **[ ]**
   - Using **expr**
   - Using **bc**
   - Let’s use these practically by terminal.
- **First** we use **let** by this command
  let RESULT=5+5
  echo The value from let : $RESULT
- Save this script and execute the file to see the result.

- **Second** we use a **(( ))** by this command.

  RESULT=$((50-25))

  echo “there result from (( )) is” : $RESULT
- And you can multiply the value by using ***** and now Save the file and execute the file to see the result.
- **Third** we use a [ ] by this command.

  RESULT=$[50/25]

  echo “The value from [ ] is :” $RESULT
- Save and execute the file.
- **Fourth** we use an **expr** by this command.

  RESULT=$(expr 300 + 200)

  Echo “The value from expr is : ” $RESULT
- Save and execute the result.
- **Last** method is **bc** by this command.

  RESULT=’echo 800 + 200 | bc ’

  Echo “The value from bc is : “ $RESULT
- Save the script and execute the file.

- **Project create a calculator using bash :**

```
#!/bin/bash
echo -n “[ + ]Enter the first number: “
read FIRST_NUM #first variable
```

```
echo

echo -n “[ + ]Enter the second number:”
read SECOND_NUM #Second variable
echo -e “\t” “press + for addition”
echo -e “\t” “press - for subtraction
echo -e “\t” “press * for multiplication
echo -e “\t” “press / for division

#Blank line
echo

echo -n “[ + ]Enter What do you want to
perform: “
read CAL #Third variable
RESULT=`expr $FIRST_NUM $CAL
$SECOND_NUM`
echo #blank line

echo “The result is:” $RESULT
```

- Save the script and execute it.

9. **STRING IN BASH :**
    - Bash **string** is a data type such as an integer or floating point unit. It is used to represent text

rather than numbers. It is a combination of a set of characters that may also contain numbers.

- **Topics Discussed :**

  Basics
  Case Modification
  Substring Removal
  Search and replace
  Finding String Length
  Substring Expansion

- Let me show you the basic difference between a variable and a string. So first we gonna use a variable like: **VAR=hello**
  echo $VAR
- If you got the output of VAR means good or otherwise showing any error means there is a string.
- Now I'm going to write the value of a string. Hello World. Like: **STR1=”Hello World”**
- And i will call it by this **echo $STR1**
- So that you can see the string value is showing.
- In this string we can use a substitution like:
  STR1=’HELLO WORLD’
  echo ${STR1/WORLD/world}
- The result will be come like this: **HELLO world**
- If you want to slice the name of character like:
  STR1=’HELLO WORLD’
  echo ${STR1:0:2}

- The output will be **HE .**Because it will slice the string.
- The 0 is first in bash.

10. **Case Modification :**
    - Case modification means that from lowercase to uppercase letters, modification of letters from Uppercase to lowercase.
    - So let create a string like: **STR1=”HELLO WORLD”**
    - **STR2=”hello world”**
    - **echo “Value from STR1:” ${STR1}**
    - And If you enter this **echo “Value from STR1:” ${STR1,,}** add a **comma.**
    - Save the file in bash and run the script.
    - It will be from uppercase to lowercase.
    - And now modify the file. change of **echo “Value from STR2:” ${STR2~~}**
    - Use this **~~** symbol from lowercase to uppercase. Like: **,, - For lowercase**
      ~~ - For Uppercase
    - Save the file and run to see the output.

11. **Substring Removal :**
    - Here we are going to learn about substring removal from the beginning. So the basic syntax

of substring from beginning is this dollar symbol $, then two brackets, then the parameter name.

- Like: **#!/bin/bash**
- **STR1=”I live in india and i am now in my home”**
- **echo ${STR1##*in}**
- It will be removed from the **beginning**.
- Now we are trying to **end removal.**
- Write the same script like:
- **#!/bin/bash**
- **STR1=”I live in india and i am now in my home”**
- **echo ${STR1%in*}**
- It will be removed after the **in** part.

12. **Search and Replace In String :**
    - Here we are going to learn about basic search and replace in bash script. Which is performed as an array.
    - like : **#!/bin/bash**
    - **STR1=”Welcome to bash script, do you like bash”**
    - **echo ${STR1/bash/python}**
    - Save and run the script.
    - The **/** will replace the word from other to another which you want.

- If you want to change both name which is same then do double // like: **echo ${STR1//bash/python}**

13. **Finding String Length :**
    - Here we are going to learn how to find string length means in a variable how many characters are there, so first of all create a bash file like:
    - **#!/bin/bash**
      STR1="hello"
      echo ${#STR1}
    - And save this code in bash and run..it gives 5 answers. Because there are 5 characters in a word.

14. **Sub-string expansion :**
    - The sub-string expansion is used to remove the number of characters from the parameter. Like : “hello world” if you like to remove the hello then you can use substring expansion.
    - **#!/bin/bash**
    - **STR1=”Welcome to bash script”**
    - # The STR1 is parameter and 7 is OFFSET (Remove number of characters)
    - **echo ${STR1:7}**
    - Save the script and run.

- **Method 2:** if you give which one you want to be print then it will be print by like this
- **echo ${STR1:10:11}**
- It will be print a **bash script**

15. **Concatenation :**
    - If you want to join the string in one line. Then use it.
    - **#!/bin/bash**
    - **STR1=”Hello, how are you”**
    - **STR2=”Hope you're good.”**
    - **echo ${STR1}${STR2}**
    - Save the script and run.

16. **Arrays :**
    - An **array is a systematic arrangement** of the same type of data. But in shell script Array is a **variable** which contains **multiple values** that may be of **same type or different types** since by default in shell script everything is treated as a **string**. An array is zero-based like: indexing starts with 0. The normal life starts from 1 but the in program starts from 0.
    - **${ARRAY[0]}** #First item of array
    - **${ARRAY[1]}** #Second item of array
    - **${ARRAY[2]}** #Third item of array
    - **${ARRAY[@]}** #All items of array

- **${ARRAY[*]}** #All items of array
- **${!ARRAY[@]}** #All indexes in an array
- **${#ARRAY[@]}** #No, of items in an array (To count the number of items in an array).
- **${#ARRAY[0]}** #Length of item zero (To get a specific length of an item).

17. **Arrays in bash :**
    - **#!/bin/bash**
    - **ARRAY=(one two three four five)**
    - **echo $ARRAY**
    - Save the script in a bash and run. It will give one string only, if you want all string then enter this line **echo ${ARRAY[@]}**
    - It will be print all string
    - If you want only one string like: one then you have to enter line **echo ${ARRAY[0]}** it will print one string only. If you give **echo {ARRAY[1]}** it will print the second item of the array.
    - And if i want to index of all item then i will enter this line **echo ${!ARRAY[@]}**
    - And if you want to know how many items are in array then enter this line **echo ${#ARRAY[@]}**
    - And if you want to specify the length of one item like: how many characters in one. Then enter this line **echo ${#ARRAY[0]}**

18. **Associative array :**
    - Here we are going to learn about associative arrays. In this array we just store a value two values in one item.
    - **#!/bin/bash**
    - **declare -A NAME**
    - **NAME=([Peter]=1000 [John]=1100 [Harry]=1200)**
    - **echo ${NAME[@]}**
    - **NAME+=([Ginney]=1300)**
    - **echo ${NAME[@]}**
    - **unset NAME**
    - **echo ${NAME[@]}**
    - Save and run the script. Then you will be get the answer the value of name
    - If you want to print the name only then you have to write **echo ${!NAME[@]}**
    - If you want any one of the item print in screen then **echo ${NAME[Peter]}**
    - If you want another name in bash then **Name+=([Ginney]=1300)**
    - **echo ${NAME[@]}**
    - **unset NAME**
    - **echo ${NAME[@]}**
    - The **unset** will work like if you wrote the same string and want to run in one time not twice then use **unset.**

19. **Indices Array :**
    - Here we are going to learn about indices in array
    - So I have created a named array and I have put six elements in it from 0 to 5.
    - **#!/bin/bash**
    - **ARRAY=(zero one two three four five)**
    - **echo ${ARRAY[@]}** #it will print all. If you want to print a particular array like 2 to 4. Then
    - **echo ${ARRAY[@]:2:3}**
    - It will be printed from 2 to 3. Save and run the script.

20. **Project on arrays :**
    - **Name=(peter john ric harry ginny)**
    - **NUMBER=(4545 5454 6565 7575 8585)**
    - **for i in {0..4}**
    - **do**
    - **echo NAME:${NAME[i]} CODE:${NUMBER[i]}**
    - **done**
    - Save the script and run in the terminal.

21. **Loop In Bash :**
    - Here we are going to learn about loops. A bash for loop is a bash programming language statement which allows code to be repeatedly executed. A loop is classified as an iteration

statement i.e. it is the repetition of a process within a bash script.

- So let's see the syntax of the loop.
- **for arg in [list]** - for loop is started from **for** and then this variable name, which is **arg,** then **in** then the list of commands or items to iterate.
- **do** - Then **do** keywords like previously we have also started in my loop.
- **command(s)...** - Then the **command.**
- **done -** Then **done** keywords to just close it.
- **Example : for arg in One Two Three..n**

```
do
echo $arg
done
```

- Let's create a basic loop. Like:
- #!/bin/bash
- for **i** in one two three four five
- do
- echo **$i**
- done
- Save the file and run. To see them there will print them one by one.
- And if I enter these in a i.
- #!/bin/bash
- for **i** in $1 $2 $3 $4 $5
- do
- echo **$i**

- done
- It will be print want you like if you save the file and while run the script then just enter something before execute like: **(./loop.sh hello)**

22. Reading files in for loop :

- Here I'm going to show you how to give a range and print that range inside a **for loop.**
- first i enter this command {1..20}
- **#!/bin/bash**
- **for i in {1..20}**
- **do**
- **echo $i**
- **done**
- Save this file and run. It will print a 1 to 20 range.
- There you can be add a file name in command substitution like : for i in **`cat /etc/passwd`**

23. C-style for loop i bash :

- Here I'm going to show you how to create a c-style i for loop. So we are familiar with the c style for loop the while loop section. So I know that you also know for loop syntax.
- So for loop like c style, so you have to write a **for** and then double parentheses **(( )) ,** you have to write your condition variable like this :

- **#!/bin/bash**
- **for ((i=1;i<=100;i++))**
- **do**
- **echo $1**
- **done**
- Save the script and run. Here we done so first, what will happen that the value of **i** will be one and i will be just equal by this echo and then after the first loop has completed the value become two and let’s check it is less than 100 so it will be rerun the loop and like that it will run until 100 it becomes hundred.

24. Infinite loop :

- Here I'm going to show you how to create an infinite for loop. So for we have to just write here **for** and the inside **double parentheses**, we have to double **semicolon ;; . then** write **for keyword**. Like:
- **#!/bin/bash**
- **count=1**
- **for ((;;))**
- **do**
- **echo $count**
- **((count++))**
- **sleep 1**
- **done**

- Save the file and run it.

25. **For loop with array :**
    - Here I'm going to use for loop with arrays. We have studied arrays. Let’s start.
    - **!#/bin/bash**
    - **ARRAY=(Peter Vivek John Harry Ginny)**
    - **COUNT=1**
    - **for i in ${ARRAY[@]}**
    - **do**
    - **echo $COUNT.$i**
    - **let COUNT++**
    - **done**
    - It will print one by one in a line.
    - You may add **echo $coun.$i** it will print with a number.
    - This is a for loop with an array.

26. **For loop with two parameters :**
    - Here we are going to learn about the use of all parameters in each list.
    - So normally you can only use one value in a list.
    - But in this process you can use two values like two parameters in each list.
    - **#!/bin/bash**
    - **for PLANET in “mercury 10” “venus 20” “earth 30” “mars 40”**

- **do**
- **set -- $PLANET**
- **echo $1 is $2,000,00 million km away from sun**
- **done**
- There I entered **set -- $PLANET** command and after that echo command is the variable first value **$1 mercury** and the dollar two will be 10. Okay , so the mercury is like 10 million kilometers away from the sun.
- Save and exit the script. Now run the script.

27. For loop with parameterized file list :

- In this video we are going to show you how to use a for loop with a parameterized file list. So like we have all used this as a symbol, we will use this in these notes.
- **#!/bin/bash**
- **for i in *.txt**
- **do**
- **echo [-] deleting $i**
- **rm $i**
- **done**
- Here i write the *.txt means it will select all txt files in directories and after the rm $i means it will remove the all .txt file from location.

28. **Nested for loop and break and continue :**
    - So here we are going to show you nested for loop and effect of break and continue conditions on for loop.
    - So we will just write a simple for loop.
    - **#!/bin/bash**
    - **for i in {1..10}**
    - **do**
    - **if [$i -le 10]**
    - **then**
    - **if [$i -eq 3]**
    - **then**
    - **continue**
    - **elif [$i -eq 6]**
    - **then**
    - **continue**
    - **elif [$i -eq 9]**
    - **then**
    - **break**
    - **fi**
    - **fi**
    - **echo $i**
    - **done**
    - Save the file and run.

29. **Real world project on for loop :**

- Here we are going to create a for loop project and understand it better.
- **#!/bin/bash**
-
- **#1. Check for root user**
- **#2. Check for Internet**
- **#3. Installing packages**
-
- **if [$UID -ne 0]**
- **then**
- **echo "Please run the script as root user"**
- **exit 1**
- **fi**
-
- **# Internet Checker**
-
- **ping -c 1 google.com &> /dev/null**
-
- **if [$? -ne 0]**
- **then**
- **echo "you don't have active internet connection"**
- **exit 2**
- **fi**
-
- **#---------- For loop----------**

- **for i in php jq**
- **do**
- **$i --verion &> /dev/null**
- **if [$? -ne 0]**
- **then**
- **echo [-]installing $i**
- **apt-get install $i -y &> /dev/null**
- **echo [+]$i installed**
- **# For loop**
- **if [$? -ne 0]**
- **then**
- **echo "Install $i manually or re-run the script"**
- **fi**
- **else**
- **echo "$i already installed....."**
- **fi**
- **done**
- Save the file in project.sh and read this full script and understand the script.

30. **Arguments :**
    - **Arguments** are input that are necessary to process the flow. Instead of getting input from a shell program or assigning it to the program, the arguments are passed in the execution part.
    - **Practical on Arguments :**
    - **#!/bin/bash**

- **echo $1**
- **echo $2**
- **echo $n**
- Save the script and run the file. If you run like: **./arguments.sh hello.** Then it will print everything after which you put in the shell..

31. **More on Arguments :**
    - **echo $#** **No.of arguments**
    - **echo $*** **All position parameters(As a single word)**
    - **echo $@** **All position parameters(As a separate string)**
    - **echo $0** **Name of the file**
    - **echo $1** **First argument**

    - Open the terminal and enter this.
    - **#!/bin/bash**
    - **IFS=”,”**
    - **echo “Output from \$#”:$#**
    - **echo “Output from \$0”:$0**
    - **echo “output from \$*”:”$*”**
    - **echo “output from \$@”:$@**
    - **echo “Output form \$1”:$1**
    - Save this script in a file and run the command. With **./file.sh hello world i am a hacker.** And

one more thing you can add **IFS=”,”** it will enter a comma in a script field.

32. **Special parameters in bash :**
    - **echo $? — Exit status of last command**
    - **echo $! — PID of last background task**
    - **echo $$ — PID of shell**
    - **echo $_ — Last argument of the previous command**
    - **echo $- — Prints the current set of options on your current shell.**
    - Go to the terminal and enter **echo $?** It will be zero. If you use the wrong command and then enter **echo $?** It will become another number.
    - And **echo $!** It will print the process id of command.
    - **echo $$** it will print the process id of the current shell.
    - **echo $_** it will print out the last argument of a command.
    - **echo $-** it will print the current set of options in your current shell. **himBH** H- history expansion, m- Monitor, h- hashall, B- brace expansion, i- interactive.

33. **Project on argument :**
    - **#!/bin/bash**

- **#USAGE: ./project.sh [NAME] [PHONE NO.] [AGE] [DEPARTMENT]**
- **echo NAME:$1 >> employes.txt**
- **echo PHONE NO:$2 >> employes.txt**
- **echo AGE:$3 >> employes.txt**
- **echo DEPARTMENT:$4 >> employes.txt**
- **echo >> employes.txt**
- **echo ========== >> employes.txt**
- **echo >> employes.txt**
- Save the script and run the script.

34. I/O redirection and piping :

- Every program we run on the command line automatically has three data streams :-
- **STDIN(0) - Standard input (Data provided to program)**
- **STOUT(1) - Standard output (What program prints, default to the terminal)**
- **STERR(2) - Standard error (What error message program prints, defaultly to the terminal)**
- So that the first is the **standard input** which we provide, which is denoted by zero. And it is provided by us to the **program,** program process, it finds errors. If it finds any **error**, then it through an error called **standard error** and standard

error is denoted by 2. If the program is successful, then it will throw an error and provide a **standard output** that is denoted by one.

- Examples:
- **ls > output.txt** #stdout to (file) **(**Here a program is run **hello.py** and **greater symbol** and **output.txt.** It will not print on the terminal but it's in runnable condition.)
- **ls >> output.txt** #stdout to (file), append

  (here nothing difference. Just using this command to add the script in an existing **file**. And **the example script is overwrite the file means remove the everything in the file and add the new script.**)
- **ls 2> error.log** # stderr to (file)

  (if you **run any program** and then the **error will be generated** then it is transfer to **error.log**)
- **ls 2>&1** #stderr to stdout

  (If you run a program if it **generates a** standard error then it will just throw a **standard error to standard output**.)
- **ls 2>/dev/null** # stderr to (null)

  (if you run a program and generate a **standard error** then it will be **thrown to a null (It just runs like a black hole and you throw anything here**.)
- **ls &>/dev/null** #stdout and stderr to (null)

(If you don't want to see any type of print on your screen after script then you can use this syntax.)

- **./script.sh < foo.txt** #feed foo.txt to stdin for python
  Reading input from a specific file, so let's think you have a **folder text** with which contains **employers names** and this program needs employers names to run. **What will happen ?**, you just provide here less than symbol **<** or in **order to read from a file**. **foo.txt** will feed to **standard input**, **input for python.**
- **diff <(ls -la) <(ls)** #Compare two stdout without files
  So you can just **compare** to **standard output** through a command called **diff**. In this you have to just write a **diff command**, then less than a symbol, then provide the **standard output** and write your command, it will compare to **standard output without a file**.

35. PIPING :

- It is basically sending one program output to another program as input.
- example : **cat output.txt | wc -l**
- So, here the **cat command** has generated an output and **without this pipe symbol**, this **wc -l**

command couldn't work with it. When you give the pipe symbol then only it will work.

- Another example : **cat name.txt | xargs mkdir**
- Here just create a **name.txt** file and enter some names in the file. And enter **| xargs mkdir.** To create a directory of the following name in the file.

36. Overview Of Exit Status :

- The **exit status is an integer number**. For the bash shell's purposes, a command which exits with a **zero (0) exit status** has succeeded. A **non-zero (1-255) exit status indicates failure**. If a command is not found, the **child process** created to execute it **returns a status of 127**.
- How to print exit status ? **echo $?**

Exit Code No	Meaning	Examples	Comments
1	Catchall for general errors	Let "var = 1/0"	Miscellaneous errors, such as "divide by zero" and other impermissible operations
2	Misuse of shell builtins (For Bash)	empty_function() {}	Missing keyword or command, or permission problem (and diff

			return code on a failed binary file comparison).
126	**Command invoked cannot execute**	**/dev/null**	**Permission problem or command i not an executable**
127	**Command not found**	**illegal_command**	**Possible problem with $PATH or a typo**
128	**Invalid argument to exit**	**Exit 3.14159**	**Exit takes only integer args in the range 0-255 (see first footnote)**
128 +n	**Fatal error signal "n"**	**Kill -9 $PPID of script**	**$? Return 137 (128+9)**
130	**Script terminated by control-C**	**Ctrl-C**	**Control-c is fatal error signal 2,(130 = 128 +2, see above)**
255 *	**Exit status out of range**	**Exit -1**	**Exit takes only integer args in the range 0-255**

- Try **Practical** on terminal to understand them. Like: **let var=1/0** and enter to get an error and check the status by using **echo $?**
- **Custom exit status :**
- **#!/bin/bash**
- **echo "Hello World"**
- **exit 5**

- # Remember the exit status must be in 1-255 between only. Not allowed to more than 255 **.**

- Save the file and run the script. After the script enters this command **echo $?**

37. If-else Overview :

- Here, we start a real bash scripting in the previous topics, we are just learning the concept of shell and terminal interpreter, exit status and many more things. So we are starting with ful statement and pay attention, Because these are very critical parts of a script or any kind other programming language it is also very critical part, so you should learn it carefully
- The if elif else statement in bash script allows creating conditional cases and responses to specific code results. The if conditional helps automate a decision-making process during a program.

- **Conditions :**
- Mathematical comparison
- String comparison
- File system related comparison

- **Syntax of IF-ELSE Statement :**

- **if [condition]**
- **then**
- **command**
- **else**
- **command**
- **fi**

- **Conditions :**
- There are conditions like, math like: equal to, plus, minus. This type of if-else also has a condition.
- Like : **[==]** or **[-eq]** #Equals to
- **[!=]** or **[-ne]** #Not equals to
- **[-lt]** #Less than
- **[-gt]** #Greater than
- **[-ge]** #Greater than equals to
- **[-le]** # less than equals to
- **[[=~]]** #Regex matching
- **[!expr]** #expr not equal
- **[x && y]** #AND
- **[x || y]** #OR

- **Mathematical comparison :**
- **#!/bin/bash**
- **if [$1 == 10]**
- **then**
- **echo “The first argument is equal to 10”**

- **else**
- **echo “The first argument is not equal to 10”**
- **fi**
- Save and run the file.
- If you want to enter **-eq** in change of **==** then you can.
- And now continue our project like changing the **==** to **-lt.** Then it will print **less than** the number which you give in the first argument. And if you change there **-gt.** It will print the first argument in **greater numbers**. If you enter **-ge** then it will be printed **greater than and equals to a number of values**. Same this type you can change **-le** it will be printed **less than and equals to a number**.
- You can use **two argument** in if-else like:
- **#!/bin/bash**
- **If [$# -ne 2] #(-ne is not equals to, and $# is entire arguments)**
- **then**
- **echo “This script requires at least 2 arguments”**
- **fi**
- **First=$1**
- **Second=$2**
- **let RESULT=$First+$Second**
- **echo The addition of $1 and $2 is : $RESULT**
- Save this **file** and **run**.

- **String Comparison :**
- **#!/bin/bash**
- **STR1=”Hello World”**
- **STR2=”Hello World”**
- **If [“$STR1” = “$STR2”]**
- **then**
- **echo “The both strings are same”**
- **else**
- **echo “strings are not same”**
- **fi**
- Save the file and run the script. Listen be careful you can’t use this **-eq -ne -lt -gt** operators. Because these are for only integers. You may use **=** or **!=**
- **Empty string comparison**.
- **#!/bin/bash**
- **STR=””**
- **If [-z “$STR”]** #(-z for empty and -n for not empty)
- **then**
- **echo “empty”**
- **else**
- **echo “String isn’t empty”**
- **fi**
- Save this in a file and run.

- **Create the project like user is root or not :**

- If you type **UID** in the terminal then it will show your **normal user id 1000**. If the root **user** checks their id, it will show **0**.
- Let's create :
- **#!/bin/bash**
- **if [$UID != 0]**
- **then**
- **echo "you must be root to run this script"**
- **echo "existing……."**
- **fi**
- Save the file and run.

- **File Conditions :**
- **[-e file]** #if you want to test for an existing file enabled statements, then you can use this option dash.
- **[-r file]** #if the file is readable then it is true, otherwise it will be false.
- **[-h file]** #simulink is just like a shortcut that you create on your desktop or any other directory to go to another.
- **[-d file]** #if a specific file is a directory, then it will return true, otherwise it will return false.
- **[-w file]** #if the file is writable then it will be returned true, otherwise it will be false.
- **[-s file]** #if file size is greater than zero bytes, then it will return true or it will return false.

- **[-f file]** #True if the variable holds an existing regular file
- **[file1 -nt file2]** #1 is more recent then 2.It just checks the date on which date or time it has been created.
- **[file1 -ot file2]** #2 is more recent than 1.
- **[file1 -ef file2]** #same. If the file is the same then it will be true.

- Let's execute the **conditions.**
- **#!/bin/bash**
- **if [-e file.txt]**
- **then**
- **echo "file exists"**
- **esle**
- **"File doesn't exist"**
- **fi**
- Save and run the script. If the file exists then it will be returned true.

- If you want to check the file is shortcut on the desktop or anywhere. Then it will be true.
- **#!/bin/bash**
- #before using this script, enter this code first. **(ln -s /home/window/deecodes/desktop/file.txt TXTFILE) by using this it will be create a shortcut in desktop.**

- **if [-h TXTFILE]**
- **then**
- **echo “This file is a symlink”**
- **else**
- **echo “This file isn't a symlink”**
- **fi**

- If you want to check for directories, then you have to use this script.
- **#!/bin/bash**
- **if [-d DIR]**
- **then**
- **echo “Directory”**
- **else**
- **echo “Not directory”**
- **fi**
- Save and run this script. Then it will be true if there is a directory.

- If you want to see the file size then enter this condition.
- **#!/bin/bash**
- **if [-s hello.txt]**
- **then**
- **echo “greater than 0 bytes”**
- **else**
- **echo “Less than 0 bytes”**

- **fi**
- Save the file and run the script.

- If you want to know if the file is readable or not. Then enter this condition.
- **#!/bin/bash**
- **if [-r file.txt]**
- **then**
- **echo “Readable”**
- **else**
- **echo “not readable”**
- **fi**
- Save the file and run the script.

- If you want to know if the file is executable or not. Then enter this condition.
- **#!/bin/bash**
- **if [-x file.txt]**
- **then**
- **echo “executable”**
- **else**
- **echo “Not executable”**
- **fi**
- Save the file and run the script.

- If you want to see the first file and last file. Then create a empty file like: **new.txt** and **old.txt**
- **#!/bin/bash**
- **if [new.txt -nt old.txt]** #put -ot for old file -nt for new file.
- **then**
- **echo “new.txt is a recent file”**
- **else**
- **echo “new.txt isn't recent file”**
- **fi**
- Save and run the script.

- **If-elif-else :**
- If you want to give two conditions or more than two conditions in a first statement, then you use the elif syntax.
- **#!/bin/bash**
- **echo -n “[+] enter a number:”**
- **read NUM**
- **if [$NUM == 10]**
- **then**
- **echo “the number is 10”**
- **elif [$NUM -lt 10]**
- **then**
- **echo “the number is less than 10”**
- **else**
- **echo “the number is greater than 10”**

- **fi**
- Save the file and run the script.

- **If-else ladder :**
- **echo -n “[+] Enter a number:”**
- **read NUM**
- **if [$NUM == 100] 2> /dev/null**
- **then**
- **echo “You have entered 100”**
- **elif [$NUM -lt 100] 2> /dev/null**
- **then**
- **echo “The number is less than 100”**
- **elif [$NUM -gt 100] 2> /dev/null**
- **then**
- **echo “The number is greater than 100”**
- **elif [[$NUM =~ [A-Z,a-z]]] 2> /dev/null**
- **then**
- **echo ”You have entered a alphabet you need t o enter number”**
- **else**
- **echo “you have specified a symbol please enter a number”**
- **fi**
- Save the file and run the program.

- **AND operator and OR operator :**

- Lists containing the **AND** and **OR** conditional operators are evaluated from left to right. A command following the **AND** operator **(&&)** is executed if the previous command is successful. The part following the **OR** operator **(||)** is executed if the previous command fails.
- **AND** is denoted by **&&**
- **OR** is denoted by **||**
- **For example**, to check for a directory and cd into it if it exists, use this:
- **test -d "$directory" && cd "$directory"**
- To change directory and exit with an error if cd fails, use this:
- **cd "$HOME/bin" || exit 1**

- **Project on If-Else : P-1**
- **!#/bin/bash**
- **echo -n "[+] Enter the domain name:"**
- **read DOMAIN**
- **ping -c 1 $DOMAIN &> /dev/null**
- **if [$? -eq 0]**
- **then**
- **echo "The $DOMAIN is up"**
- **else**
- **echo "The $DOMAIN is down"**
- **fi**
- Save the file and run the script.

- **Project on If-Else : P-2**
- **#!/bin/bash**
- **echo -n “[+] Enter the file name:”**
- **read FILE**
- **READ=0**
- **WRITE=0**
- **EXECUTE=0**
- **if [-r $FILE]**
- **then**
- **READ=YES**
- **else**
- **READ=NO**
- **fi**
- **if [-w $FILE]**
- **then**
- **WRITE=YES**
- **else**
- **WRITE=NO**
- **fi**
- **if [-x $FILE]**
- **then**
- **EXECUTE=YES**
- **else**
- **EXECUTE=NO**
- **fi**
- **clear**
- **echo ========++++++========**

- **echo “Checking for file permissions”**
- **echo ========++++++==========**
- **echo “FILE:$FILE”**
- **echo READABLE:$READ**
- **echo WRITABLE:$WRITE**
- **echo EXECUTABLE:$EXECUTE**
- **echo ==========**
- Save the file and run the script.

38. Regular Expressions :

- A regular expression is some sequence of characters that represents a pattern. For example, the **[0-9]** in the example above will match any single digit where **[A-Z]** would match any capital letters. **[A-Z]+** would match any sequence of capital letters.
- **Note: it matches only text not files.**
- There are two types of regular expressions.
- **BRE** (Basic regular expressions)
- **ERE** (Extended regular expressions)

- **ERE syntax:**
- If you are using the ERE, then you have to use options like in these tablets.

Special characters	Functions
.	Matches one character
[ ]	Character sets
\	Escape single character
( )	Pattern grouping
|	Alternation

- **BRE syntax :**
- So if you are using BRE it means basic regular expression. Like in these tablets.

Special characters	Functions
\.	Matches one character
\[\]	Character sets
\\	Escape single character
\(\)	Pattern grouping
\\|	Alternation

- REGEX support in GNU tools.
-

Tools	Regex type
grep -G	BRE

grep -E	ERE
sed	BRE
sed -E	ERE
awk (only support ERE)	ERE
[[=~]]	ERE

-

Symbol	Description	Example	Matches
.	Any single character	bo.k	Book, bolk
*	Preceding character must match zero or more times	co*l	col,cool
∧	Start of the line marker	∧hello	Line starting with hello
$	End of the line marker	hello$	Line ending with hello
[ ]	Any one of character enclosed in [ ]	coo[k]	cook or cool

[∧]	Any character except those enclosed in brackets.	file[∧123456]	file7,file8
[-]	Any character within range	file[1-3]	file1,file2,file3
?	Preceding item must match one or zero item	ab?c	ac OR abc NOT abbc
+	Preceding item much matches one or more time	file1+	file1,file11,file111
{n}	Preceding item much matches n times	[0-9]{3}	Any three digit number
{n,m}	Min and max of times preceding item should match	[0-9]{1,3}	1,154,34 note-1111,2000
\	The backslashes escape token	a\\t	a\\t MATCHES a\t

	characters in order to match those characters literally		a\\|b MATCHES a|b
()	Group patterns	a(b|c)	abd and acd

- Practical on regular expression :
  - **Cat file.txt | grep bo.k**
  - It will return book and bolt. If there is a file.
  - Try on your own mind by creating a file.

- **Introduction to character class :**
- Here, we match a digit from the file by using **[[:digit:]]**
- Example:
- **cat test.txt | grep [[:digit:]]**
- **1234**

- **Character classes** :

Class	Match
[[:digit:]]	**[0-9]**
[[:lower:]]	**[a-z]**
[[:upper:]]	**[A-Z]**

[[:alpha:]]	[a-zA-z]
[[:alnum:]]	[0-9a-zA-Z]
[[:xdigit:]]	[0-9a-fA-F]
[[:cntrl:]]	Control char- first 32 ASCII characters and 127th(DEL)
[[:punct:]]	All the punctuation characters
[[:graph:]]	[:alnum:] and [:punct:]
[[:print:]]	[:alnum:],[:punct:] and space
[[:blank:]]	Space and tab characters
[[:space:]]	Whitespace characters, same as \s

- Project on regular expression :
- #!/bin/bash
- # Project for valid IP address checker
- #192.168.543.345
- echo -n “[+] Enter the ip address:”
- read IP
- REGEX=”[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[[:digit:]]{1,3}”
- if ! [[$IP =~ $REGEX]]
- then
- echo “Not a valid ip address”
- fi

- **IP_ADDRESS=${BASH_REMATCH[0]}**
- **echo $IP**
- **clear**
- **ping -c 1 $IP_ADDRESS &> /dev/null**
- **echo ==========**
- **echo YOUR IP:$IP_ADDRESS**
- **echo ==========**
- **if [$? -eq 0]**
- **then**
- **echo “IP ADDRESS IS ALIVE”**
- **else**
- **echo “IP ADDRESS IS DEAD”**
- **fi**
- Save this file and execute the script.

- **Task on regular expression :**
- In this session match the credit card number. This is the credit card like : 1234-1234-1234-1234
- **cat card.txt | grep -E “[0-9]{4}\-[0-9]{4}\-[0-9]{4}\-[0-9]{4}”**

39. **Globing :**
    - Globing is nothing but a way to match files in linux.
    -

Regular expressions	Glob

It matches text	It matches files
ls | grep [0-9]file*\.txt	ls [0-9]?file*.txt

- Shell Glob options.
- **nullglob**
- **failglob**
- **dotglob**
- **globstar**
- **nocaseglob**
- **globstar**
- **extglob**

- **Setting shell options** :
- **shopt -s globname** —> set it
- **shopt -u globname** —> unset it
- **shopt -p** —> print out the current value

- **Practical on glob** :
- First create some empty files, and otherwise use a small script with glob like:
- **!#/bin/bash**
- **shopt -s nullglob**
- **FOUND=0**
- **for i in *.jpg**
- **do**
- **echo $i**

- **done**
- Save this fail and run, if there is no error and script runned. Then the nullglob will work..!!! And create an error to see your project is really working.
- Now use the same script but change the **nullglob to failglob.** Then, it will show the error in the script.
- **shopt -s dotglob.** It will show hidden files. Nothing just go terminal and enter the command **shopt -s dotglob** and **echo *** and run the command to see the hidden files will be shown in output.
- **shopt -s globstar.** It will show the files directory contains files. Enter this command **shopt -s globstar** and **ls *** and run the command to see the directories contain files.
- **shopt -s nocaseglob.** It just print the uppercase and lowercase by typing **ls.** Like : **shopt -s nocaseglob** and **ls [ab]*** and run the command to see the print.

- **Extended Globing :**
- Why you should use extended glob ? Make interactive globing more useful, files that starts either **IMG or DSC but do not end with .CRW or .NEF**

- **Advantages of extended glob**
- Patterns can be more than one character
- Multiple match occurrence
- Allow grouping
- Nesting pattern groups
- Logical AND or OR
- Add power to if conditional statement
- Add power to case statement
- Add power to bash pattern substitution
- Speed

- **Examples :**
- **@(MATCH)** —> matches only one occurrence
- **?(MATCH)** —> matches 0 or 1 occurrence
- **+(MATCH)** —> Matches more than 1 occurrence
- ***(MATCH)** —> matches 0 or more occurrence
- **!(MATCH)** —> matches no occurrences
-

Pattern	Match	Would not match
photo@(.jpg)	photo.jpg	Photo photo.jpg.jpg
photo?(.jpg|.png)	Photo.jpg, photo.png photo	Photo.gif, photo.png.png, photo2.png

photo+(.jpg|.png)	Photo.jpg, photo.jpg.png	Photo Photo.gif, photo2.gif
photo*(.jpg|.png)	Photo.jpg, photo.jpg.png Photo photo.png	Photo.jpg, photo.gif, photo2.gif
photo!(.jpg|.png)	Photo Photo.gif, photo.bmp, photo.txt	Photo.jpg, photo.png
photo!(?(.jpg|.png)	Photo, photo.gif, photo.bmp, photo.txt	Photo.jpg, photo.png
!(+(photo|file)*+(.jpg|.gif))	All files that do not have a photo or file and doesn't end with .jpg or .gif	

- Try on your own example.

- **Difference between regular expression and globing :**
- You can test the glob by using for loop script.
- **#!/bin/bash**

- **for i in ***
- **do**
- **if [[$i = *(*.tar.gz)]]**
- **then**
- **cp $i /home/deecodes/**
- **fi**
- **done**
- Save this and run the command **“time ./glob.sh”**

- You can test the glob by using for loop script.
- **REGEX=.tar.gz**
- **for i in ***
- **do**
- **if [[$i =~ $REGEX]]**
- **then**
- **cp $i /home/deecodes**
- **fi**
- **done**
- Save this script and run the file by this command **time ./regular.sh**

40. While loop in bash :

- The condition for a while loop is a list of one or more commands, and the commands to be executed while the condition remains true are placed between the keywords **do** and **done.**

- **Example :**
- **while <list>**
- **do**
- **<list>**
- **done**

- **Lets try a project on while loop :**
- **#!/bin/bash**
- **LIMIT=1**
- **while [$LIMIT -lt 10]**
- **do**
- **echo $LIMIT**
- **let LIMIT++**
- **done**
- Save the file and run this script.
- The script will do..There is a variable **LIMIT.** The variable will be **moved in while** and, while will **check the integer** is **less than 10** or not. And after this it will go to **let** and let will **add 1** and loop the script **until greater than 10 number**. It will **add 1** in **LIMIT variable**.

- **C-style while loop :-**
- **#!/bin/bash**
- **LIMIT=1**
- **while (($LIMIT < 10))**
- **do**

- **echo $LIMIT**
- **let LIMIT++**
- **done**
- Save and run the script.

- **Reading line from while loop :-**
- **#!/bin/bash**
- **NUM=1**
- **cat names.txt |**
- **while read line**
- **do**
- **echo $NUM.$line**
- **let NUM++**
- **done**
- Save and run. It will **print the name with numbers**.

- **Infinite in while loop :-**
- **#!/bin/bash**
- **VAR=1**
- **while true**
- **do**
- **echo $VAR**
- **done**
- Save and run the file. It will run infinitely until you stopp…!!!!

- **While loop with multiple conditions :-**
- **#!/bin/bash**
- **VAR1=unset**
- **PREVIOUS=$VAR1**
- **while echo “Previous-variable=$PREVIOUS”**
- **echo**
- **PREVIOUS=$VAR1**
- **[“$VAR1” != end]**
- **do**
- **echo -n “[+] Input variable”**
- **read VAR1**
- **echo “Variable #1=$VAR1”**
- **done**
- **exit 0**
- Save and run the file.

- **Nested while loop :-**
- It is just a combination of an if-else statement, while loops and different kinds of things.
- **#!/bin/bash**
- **TABLE=1**
- **echo -n “[+] Enter the number the script will create a table:”**
- **read NUM**
- **If [[$NUM == [[:digit:]]]]**
- **then**
- **while [$TABLE -le 10]**

- **do**
- **echo $NUM*$TABLE | bc**
- **let TABLE++**
- **done**
- **else**
- **echo “you have entered a invalid character, you have entered $NUM”**
- **fi**
- Save the script and run.

- **Break and continue with while loop :-**
- While continue causes a jump to the next iteration of the loop, skipping all the remaining commands in that particular loop cycle.
- The break and continue loop control commands correspond exactly to their counterparts in other programming languages. The break command terminates the loo[(breaks out of it)
- **#!/bin/bash**
- **LIMIT=0**
- **while [$LIMIT -le 10]**
- **do**
- **let LIMIT++**
- **if [$LIMIT -eq 5]**
- **then**
- **break**
- **else**

- **echo $LIMIT**
- **fi**
- **done**
- Save and run the script. The script will **break** if 5 is printed in the terminal. If you wrote a **continue** then the script will be continued without 5.

- **Project on while loop :-**
- **#!/bin/bash**
- **FILE=$1**
- **cat $FILE |**
- **while read line**
- **do**
- **ping -c 1 $line &>/dev/null**
- **if [$? -eq 0]**
- **then**
- **echo $line is up**
- **else**
- **echo $line is down**
- **fi**
- **done**
- Save and exit the file. To see the command how it will be executed.

- **Task on while loop (test) :-**
- **#!/bin/bash**
- **echo -n “[+] Enter your password:”**

- **read PASSWD**
- **if [“$PASSWD” = “12345”]**
- **then**
- **echo Welcome `whoami`**
- **else**
- **while [“$PASSWD” != “12345”]**
- **do**
- **echo -n “[+] Enter the correct password:”**
- **read PASSWD**
- **if [“PASSWD” = “12345”]**
- **then**
- **echo “Welcome `whoami` have entered correct password”**
- **fi**
- **done**
- **fi**
- Save the file and run it.

41. **Until loop :-**
    - **#!/bin/bash**
    - **NUM=1**
    - **until [$NUM -gt 10]**
    - **do**
    - **echo $NUM**
    - **let NUM++**
    - **done**
    - Save and run the script to check if it works or not.

42. Case :-

- The case construct is the shell scripting analog to switch in C/C++. It permits branching to one of a number of code blocks, depending on condition tests. It serves as a kind of shorthand for multiple if/then/else statements and is an appropriate tool for creating menus.
- **Example :**

```
case “$variable” in
        “$condition1” )
                command…
        ;;
        “$condition2” )
                command…
       ;;
esac
```

- **Syntax and basic of case :-**
- **#!/bin/bash**
- **case `whoami` in**
- **"root")**
- **echo "You are root ....."**
- **;;**
- **"deecodes")**
- **echo "You are Deecodes user....."**
- **;;**
- ***)**

- **echo "You are someone else"**
- **esac**
- This is an example of a case. Save and run it. If you are in root then it will be executed or otherwise it will say you are a other user.

- Let's check a punctuation from case :
- **#!/bin/bash**
- **echo -n “[+] Enter the character”**
- **read CHAR**
- **case “$CHAR” in**
- **[[:lower:]])**
- **echo “Lower case character…”**
- **;;**
- **[[:upper:]])**
- **echo “Uppercase character…”**
- **;;**
- **[[:digit:]])**
- **echo “It is a digit…”**
- **;;**
- ***)**
- **echo “It is a punctuation”**
- **esac**

- Save and run the script. Like if it was lower, upper or any digit and any word. Then it will be their output.

- **More on case :-**
- Here we are learning about whether we will just print out three people's names and if we select any one of them, then it will display a specific person.
- **#!/bin/bash**
- **echo ==========**
- **echo Enter the name of person you want to get details..**
- **echo ==========**
- **echo 1.Peter**
- **echo 2.Vivek**
- **echo 3.Harry**
- **echo —----------**
- **echo -n "Enter here:"**
- **read NAME**
- **echo —----------**
- **case "$NAME" in**
- **"Peter" | "peter")**
- **echo NAME:Peter Parker**
- **echo PHONE: 456445445**
- **echo G-Mail: Peter@gmail.com**
- **echo DEP: Red teamer**

- **echo —----------**
- **;;**
-
- **“Vivek” | “vivek”)**
- **echo NAME:Vivek Parker**
- **echo PHONE: 456545445**
- **echo G-Mail: Peter@gmail.com**
- **echo DEP: Blue teamer**
- **echo —----------**
- **;;**
-
- **“Harry” | “harry”)**
- **echo NAME:Harry Parker**
- **echo PHONE: 456745445**
- **echo G-Mail: Peter@gmail.com**
- **echo DEP: Grey teamer**
- **echo —----------**
- **;;**
- **esac**
- Save this script and run to try and understand about this script.

- **Create menu using “select” :-**
- **#!/bin/bash**
- **PS3=’[+] Enter Your Fav Programming Language:’**

- **select LANG in “Bash” “Python” “HTML” “Go” “Java” “CSS” “AWK” “Dart” “Flutter” “Javascript”**
- **do**
- **echo**
- **echo “Your fav programming language is $LANG”**
- **break**
- **done**
- Save and run the script. It will be use to select the string.

- **Rechecking user input using case :-**
- **clear**
- **echo -n “Enter Your Full name:”**
- **read NAME**
- **echo -n “Enter Your Department”**
- **read DEP**
- **echo -n “Enter Your Age”**
- **read AGE**
-
- **echo -n “Do you have entered correct info [y/n]:”**
- **read ANS**
- **case “$ANS” in**
- **“Y” | “y”)**
- **echo $NAME >> emp.txt**

- **echo $DEP >> emp.txt**
- **echo $AGE >> emp.txt**
- **echo —----------**
- **;;**
- **“N” | “n”)**
- **echo “Nothing is saved in the file”**
- **;;**
- ***)**
- **echo “Something wrong”**
- **;;**
- **esac**
- Save the script and run. It will be save the info and if the wrong then it will be not save by using case. This is rechecking format.

- **Creating a script with parameter :-**
- Here we are using the case to help section. Like : **./case.sh --help**
- **#!/bin/bash/**
- **while [$# -eq 0]**
- **do**
- **echo “Usage: [-h| --help] [-f] file [--file] file”**
- **;;**
- **while [$# -gt 0]**
- **do**
- **case “$1” in**
- **“-h” | “--help”)**

- **echo “Usage: [-h| --help] [-f] file [--file] file”**
- **exit 1**
- **;;**
- **“-f” | “--file”)**
- **FILE=$2**
- **LINE=`cat $FILE | wc -l`**
- **WORD=`cat $FILE | wc -w`**
- **LETTER=`cat $FILE | wc -m`**
- **clear**
- **echo----------**
- **echo FILE:$FILE**
- **echo —----------**
- **echo No. of lines: $LINE**
- **echo No. of words: $WORD**
- **echo No. of letter: $LETTER**
- **echo —----------**
- **exit 2**
- **;;**
- ***)**
- **echo “Usage: [-h| --help] [-f] file [--file] file”**
- **exit 3**
- **;;**
- **esac**
- **done**
- Save the script and run. In this script we use the tool in one line command. **By using a case.**

- **Real world project on case :-**
- In this project we grab the information of our machine by using case.
- **#!/bin/bash**
- **while [$# -gt 0]**
- **do**
- **case “$1” in**
- **“-h” | “--help”)**
- **echo Usage: $0 [Options…]**
- **echo Options:**
- **echo -e “\t” “-c , --cpuinfo”**
- **echo -e “\t” “\t” “\t” “Get CPU information”**
- **echo -e “\t” “-k , --kernal-info”**
- **echo -e “\t” ”\t” ”\t” “Get Kernal Information”**
- **echo -e “\t” “-s , --sys-info”**
- **echo -e “\t” ”\t” ”\t” “Get system Information”**
- **echo -e “\t” “-fm , --free-mem”**
- **echo -e “\t” ”\t” ”\t” “Get free space Information”**
- **echo -e “\t” “-ip , --ipaddr”**
- **echo -e “\t” ”\t” ”\t” “Get IP address Information”**
- **echo “Created by Deepak Choudhary”**
- **exit 1**
- **;;**
- **“-c” | “--cpuinfo”)**

- CPU=`cat /proc/cpuinfo | grep “model name” | cut -c 14-50`
- echo CPU INFO: $CPU
- echo
- shift
- ;;
- “-k” | “--kernal-info”)
- echo KERNAL VERSION:`uname -r`
- echo
- shift
- ;;
- “-s” | “--sys-info”)
- SYSINFO=`cat /etc/os-release | grep “PRETTY” | cut -c 13-50
- echo SYS-VER:$SYSINFO
- echo
- shift
- ;;
- “-fm” | “--free-mem”)
- FREE=`free -m | grep Mem | awk '{print $4}'`
- echo FREE SPACE: $FREE mb
- echo
- shift
- ;;
- “-ip” | “--ipaddr”)
- echo IP ADDRESS: `hostname -I`
- echo

- **shift**
- **;;**
- **esac**
- **done**
- Save and run the script it will be used in one line command.

43. **Function :-**
    - **Introduction to functions in bash :-**
    - The function is a very important part. Function is a subroutine, a code block that implements a set of operations, a “black box” that performs a specified task. Wherever there is repetitive code, when a taks repeats with only slight variations in procedure then consider using a function.
    - **Way to create functions. Example :**
    - **Method 1 :-**
    - function function_name {
    - command…
    - }
    -
    - **Method 2 :-**
    - function_name () {
    - command…
    - **}**
    -
    - **Syntax of functions :-**

- **#!/bin/bash**
- **#method 1**
- **function hello {**
- **echo “Output from method one”**
- **}**
- **hello**
- **#method 2**
- **world () {**
- **echo “Output from method 2”**
- **}**
- **world**
- Save the script and run. Hello will be work like a variable.

- **More on functions :-**
- **#!/bin/bash**
- **function usage {**
- **echo "usage: $0 [-h|--help] [-f file] [--file file]"**
- **exit 1**
- **}**
-
- **while [$# -gt 0]**
- **do**
- **case "$1" in**
- **"-h" | "--help")**
- **Usage**

- **;;**
-
- **"-p" | "--path")**
- **cd $2 ; ls -l |**
- **while read line**
- **do**
- **TIME=`echo $line | awk '{print $8}'`**
- **FILE=`echo $line | awk '{print $9}'`**
- **echo File : $FILE was created on: $TIME**
- **done**
- **exit 1**
- **;;**
- ***)**
- **echo "usage: $0 [-h|-help] [-f file] [--file file]"**
- **exit 1**
- **;;**
- **esac**
- **done**
- Save the script and run. It will be read the lines and grep the time date and folder name.

44. Cron jobs :-

- Cron is one of the most useful that you can find in any unix-like operating system. Cron is used to schedule commands at a specific time. These scheduled commands or tasks are known as “Cron Jobs”. Cron is generally used for running

scheduled backups, monitoring disk space, deleting files periodically which are no longer required, running system maintenance tasks and a lot more.

- **Symbols used in cron jobs :-**
- **SYMBOLS MEANING**
- *** Any value**
- **, Value list separator**
- **- Range of values**
- **/ Step values**
- There is a 5 * in cron jobs like ***** **<Command to execute>**
- First * is denote **min (0-59)**
- Second * is denote **Hours (0-23)**
- Third * is denote **day of month (1-31)**
- Fourth * is denote **month (1-12)**
- Fifth * is denote **day of week (0-6) 6 or 7 are sunday.**
- To display the contents of the crontab file of the currently logged in user: **crontab -l**
- To edit the current user's cron jobs, do: **crontab -e**
- To edit the crontab of a different user, for example deecodes, do: **crontab -u deecodes -e**
- To remove all cron jobs for the current user: **crontab -r**

- Example of crontabs :-

Cron jobs	Meaning
*****<command-to-exe>	Every minute
*/5****<command-to-exe>	Every 5th minute
*/15****<command-to-exe>	Every quarter hour
30****<command-to-exe>	Every hour at minute 30
0,5,10****<command-to-exe>	Three times every hour, at minute 0,5 and 10:

- **Non standard cron jobs :-**
- **Cron job strings** **Action**
- @reboot run once, at startup
- @yearly run once a year.
- @annually (same as @yearly)
- @monthly run once a month
- @weekly run once a week
- @daily run once a day
- @midnight (same as @daily)
- @hourly run once an hour

- **More on cron job :-**
- **crontab -h** to get help section. From **crontab -e** to edit the cronjob. After enter this command give the execution command like: ***** **cd /home/deecode ; touch hello.txt**

- **Save** and wait for a few minutes. It will be created at that time.
- We can choose a time, day and month to execute any script.
- Verify the crontab by using the website "https://crontab.guru/" it will be controlled by the star. If you enter four **stars** then **it will be shown minute**.
- If you want to use crontab in GUI mode then use this link "https://crontab-generator.org/"
- This link will generate a command line of the crontab in GUI mode.

- **Real world project on crontab :-**
- Here we create a script that will send automatic mail alerts when server memory gets low.
- Here we use **free -mt** the **free** is used to see **memory** and **m** is helpful to see **mb** and **t** is **total**.
- **#!/bin/bash**
- **FREE=`free -mt | grep Total | awk '{print $4}'`**
- **if [$FREE -le 3163] #(Enter your own memory)**
- **then**
- **touch /tmp/memory_low.txt**
- **ps aux | head > /tmp/memory_low.txt**
- **fi**

- **FILE=/tmp/memory_low.txt**
- **echo “server memory is getting low !!!” | mail -s $FREE mb left -A $FILE deecode37@gmail.com**
- Save and exit the script and enter the **crontab -e** to add the script location to automate the execution.
- *** * * * * cd /home/deecode/ ; ./script.sh**
- Save and exit the script run the script and wait for a message to mail. Create the mail in linux by own by watching videos.

45. Adding colors to shell script :-

- In this section we are learning about how to add colors in bash.
- Colorizing Shell.
- echo **-e “\e[COLORm**Sample Text**\e[0m”**

Option	Description
-e	Enable interpretation of backslash escapes
\e[	Begin the color modifications
COLORm	Color code + ‘m’ at the end

\e[0m	End the color modifications

- Color Escape codes
-

color	foreground code	Background code	sample
Black	30	40	
Red	31	41	
Green	32	42	
Brown	33	43	
Blue	34	44	
Purple	35	45	
Cyan	36	46	
Light gray	37	47	

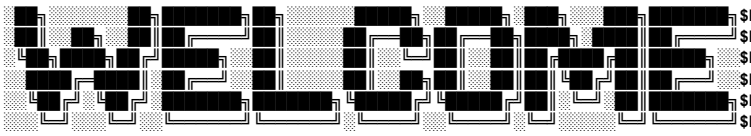
- ASNI CODE

ASNI code	Description
0	Normal Characters
1	Bold Characters
4	Underlined characters

5	Blinking Characters
7	Reverse video characters

- **Practical of color :-**
- **echo -e "\e[35m text \e[0m"**
- This is good example for **adding color in bash**.
- **echo -e "\e[5m text \e[0m"**
- This is good example for **ASNI code.**
- **echo -e “\e[1;35;47m Hello World \e[0m”**
- This is good example for **blink, bg color, and text color in one.**

- **Creating a welcome on every time you open your terminal :-**
- Open **bashrc** file, you cant see the file by **ls**. Because it was hidden file. Enter **nano .bashrc**
- Enter this at the last line.
- **RED=”\e[31m”**
- **END=”\e[0m”**
-
- **echo -e “$RED** **$END”**
- **echo -e “$RED** **$END”**
- **echo -e “$RED** **$END”**
- **echo -e “$RED** **$END”**
- **echo -e “$RED** **$END”**
- **echo -e “$RED** **$END”**



- Save and open new tab to see the color in terminal.

46. **Printf Command :-**

- “printf” command in linux is used to display the given string, number or any other format specifier on the terminal window. It works the same way as “printf” works in programming languages like C. Note: printf can have format specifiers, escape sequences or ordinary characters.
- **Syntax :**
- **printf FORMAT ARG …**

- **Printf Options :**
- **\a: Alert (bell)**
- **\b: Backspace**
- **\e: Escape character**
- **\f: Form feed**
- **\n: newline**
- **\r: Carriage return**
- **\t: Horizontal tab**
- **\v: Vertical tab**
- **\\: Backslash**

- **Format Specifier :**
- The **%s** specifier prints the literal characters in the argument.
- **$printf “%s\n”** print arguments on **“separate lines”**

- **%b** is like **%s** except that escape sequences in the arguments are translated.
- **$print “%b\n” “hello\nworld” “12\tword”**
- Integers are printed with **%d**. The integer may be specified as a decimal, octal (Using a leading 0), or hexadecimal (preceding the hex number with 0x) number. If the number is not a valid integer, printf prints an error message.
- **$printf “%d\n” 23 45 56.78 0xff 011**
- For decimal fractions or floating-point numbers, use **%f**. By default they will be printed with six decimal places
- $ **printf “%f\n” 12.34 23 56.789 1.2345678**
- Floating-point numbers can be presented in exponential (also known as specific) notation using **%e.**
- **printf “%n” 12.34 23 56.789 1.2345678**

- **Width Specification :**
- **printf “%8s %-15s:\n” first second third fourth fifth sixth.**
- First second :
- Third fourth :
- Fifth sixth :

- A width specifier with a decimal fraction specifies the precision of a floating-point number or the maximum width of a string.
- **printf “%12.4s %9.2f\n” john 2 jackson 4.579 walter 2.9**

- **More on printf command :-**
- **printf “%s\n” “Hello world”**
- This will print the string on display like an echo command. You may use a variable through to execute like : **var=”hello world”**
- **printf “%s\n” “$var”**

- **Width specification in printf command :-**
- **printf “%10s %-15s\n” Hello world**
- **printf “10s %-15s\n” One Two Three Four Five Six**
- **printf “10.2s %-15.2s\n” One Two Three Four Five Six** #if you enter .2 then your last 2 or 1 string will be removed.

- **Creating a table :-**
- **#!/bin/bash**
- **#employees data**
- **#Divider**
- **DIVIDER==========**
- **#Header**

- **printf “\n%-10s %-10s %-15s\n” NAMES DEPARTMENT SALARY**
- **printf “%s\n” $DIVIDER**
- **# Body**
- **printf “\n%-10s %-10s %-15s\n” Deepak “Red Teamer” 100000 Surabh “Blue teamer” 200000 Anirudhh “Purple teamer” 300000**
- **printf “%\ns” $DIVIDER**
- Save and exit the script and run the script. It will create a table.

47. Introduction to awk command :-

- Awk (awk) is a domain-specific language designed for text processing and typically used as a data extraction and reporting tool like sed and grep, it is a filter, and is a standard feature of most unix-like operating systems.
- **Syntax :**
- **awk ‘BEGIN{**
- **Code_in_begging_section**
- **}**
- **{Code in main body}**
- **END {**
- **Code_in_End**
- **}’ input_file-1 input_file-2**

- **Example :**
- **awk ‘BEGIN{**
- **print “1234”**
- **}**
- **{print “Hello world”}**
- **END {**
- **print “All done”**
- **}’ input_file-1 input_file-2**

- **Output:**
- **1 2 3 4**
- **Hello world**
- **All done**

- These are some things like we will use in our scripting with awk **“NF (No. of fields) NR (No of records) RS (Record Separator)”**

- **AWK practical :-**
- **awk ‘{print “Hello world”}’ file.txt**
- This script will print hello world on the terminal. On each line of the file.
- There are **3 types** of print files in awk.
- **awk ‘{print}’ file.txt**
- **awk ‘{print $0}’ file.txt**
- **awk ‘{print $_}’ file.txt**

- **awk ‘{print NF}’ file.txt** (It will be print no of field like: rows)
- **awk ‘{print NR}’ file.txt** (It will be print a no of records)
- **awk ‘{print RS}’ file.txt** (it will be print record separator)
- **awk ‘{print $1}’ file.txt** (it will print first field only. If you enter 2 then second and it will be continue)
- **awk ‘BEGIN{print “TABLE OF STUDENTS”} {print}’ file.txt** (It will be print begin at the file)
- **awk ‘BEGIN{print “TABLE OF STUDENTS”} {print}’ END{print “Table completed”}’ file.txt** (it will print the end at the file)

- **Regular expression with awk :-**
- Here we are learning about pattern matching.
- In awk if you want to print a horizontal line in a table like:
- **awk ‘/name/{print }’ file.txt**
- The **name** is one of the words from the file. To print the horizontal line.
- **awk ‘!/name/{print }’ file.txt**
- The **name** is one of the words from the file that will be removed after executing this command **“!”**
- **awk ‘!/[[:digit:]]/{print }’ file.txt**
- After entering the **[[:digit:]]** command. It will remove all the digits from the file.

- **Field separators :-**

- **awk -F : ‘{print $1 “ “ $7}’ /etc/passwd**
- After entering this you can see the **-F** is used to field and **:** is **removed** from the print and the **‘{print $1 “ ” $7}’** is used to **print 1** field and **“ ”** to **space** and **print $7** is the **seventh field**.

- **Boolean operators in awk :-**
- **awk ‘BEGIN{print 5+1}’**
- It will add the number. This type of boolean works. You may add sub or multiple and divide too.

- **If-else in awk :-**
- **#!/bin/bash**
- **BEGIN {**
- **AVG=69**
- **if (AVG >= 100)**
- **print “PASS”**
- **else**
- **print “FAIL”**
- **}**
- Save the file and execute the script it will show the fail or pass. Use command **awk -f file.awk (save the extension in awk)**

- **Loops in awk :-**
- **awk ‘BEGIN {for (i=1; i<=5; ++i) print i}’**
- You can change the number value.
- **awk ‘BEGIN {i=1; while (i <=6){print i; i++}’**
- **awk ‘BEGIN {i=1; do {print i; i++} while (i <=6)’**

48. Sed command :-

- **sed** is a useful text processing feature of **GNU/Linux**. The full form of **sed** is **stream editor**. Many types of simple and complicated text processing tasks can be done very easily by using sed commands. **Any particular string in a text or a file can be searched, replaced and deleted by using regular expression with sed command**. But this command performs all types of modification temporarily and the original file content is not changed by default. The user can store the modified content into another file if needed.
- **Syntax :**
- **sed ‘3,9d’ file.txt**
- **Commands :**
- **a text -** append text after a line
- **d -** delete the pattern space
- **i text -** insert text before a line
- **p -** print the pattern space

- **q[exit code] -** exit sed without processing any more commands or input.
- **s/regex/replacement/[flag] -** matches the regular expression and changes it.

- **More on sed command :-**
- **sed -n ‘/name/p’ file.txt**
- It will print the name line only.
- **sed -n ‘6p’ file.txt**
- It will print the 6th line of the file. If you want to add a more line to print then **‘3,6p’**
- **sed ‘/name/a hello’ file.txt**
- It will overwrite the word. **a** is an **append**. If you enter **“i”** then it will print before the line.
- **sed ‘/name/d’ file.txt**
- It will delete the name of the line.
- **sed ‘s/name/hello/’ file.txt**
- It will remove the name and print hello at the given string.
- **sed ‘s/name/hello/g’ file.txt**
- It will replace all **name words** with **hello**. By using **g.** If you enter **g2,** then the second word also changes it.
- **echo “Bash language” | sed ‘s/\(Bash\)/Learn \1 Programming/’**
- It will be entered before the bash. And programming will be at the end.

- **sed ‘/^$/d’ file.txt**
- It will remove all blank spaces.

49. **Simple projects :-**
    - **Random quote generator :-**
    - Here, I have a good command. **echo $RANDOM.** This command will create random **numbers**.
    - And one more command is **echo $((RANDOM%5))**
    - **#!/bin/bash**
    - **while read -r line**
    - **do**
    - **arr[$i]=$line**
    - **let i++**
    - **done < file.txt**
    - **RAN=$((RANDOM%i))**
    - **echo -e “\e[1;31m${arr[$RAN]}\e[0m”**
    - Save and before running the script just create a few lines of quote file. Because you have to put the file.txt into quote lines.

    - **Digital Clock :-**
    - **#!/bin/bash**
    - **while :**
    - **do**
    - **clear**
    - **toilet `date +%H:%M:%S`**

- **sleep 1**
- **done**
- Save and run the script.

- **Password generator :-**
- **#!/bin/bash**
-
- **RED="\e[1;31m"**
- **BLUE="\e[1;33m"**
- **RESET="\e[0m"**
-
-
- **case "$1" in**
- **"-h" | "--help")**
- **echo "USAGE:$0 [-h|--help] help [-c|--char] character"**
- **echo "Example: $0 -c 8"**
- **;;**
- **"-c" | "--char")**
- **PASS=`date | sha256sum | cut -c 1-"$2"`**
- **echo -e ${RED} YOUR $2 LENGTH PASSWORD IS :${RESET} ${BLUE} $PASS ${RESET}**
- **;;**
- ***)**

- **echo "USAGE:$0 [-h|--help] help [-c|--char] character"**
- **echo "Example: $0 -c 8"**
- **;;**
- **esac**

- Method 2: Add special character:
- **#!/bin/bash**
- **MATRIX="1234567890QWERTYUIOPLASD!@#$%^&"**
- **LENGTH="$1"**
- **while ["${n:=1}" -le "$LENGTH"]**
- **do**
- **PASS="$PASS${MATRIX:$(($RANDOM%${#MATRIX})):1}"**
- **let n++**
- **done**
- **echo -e "\e[1;31mYour Password Is:\e[0m \e[1;34m$PASS\e[0m"**
- Save the script and run.. It will create a password of a special character too. **Pass** are the **variable** and **$PASS** are also **pass** variables. And **random command** are also included.

50. System administration projects :-

- **Script for server backup:**

- You need one more kernel. You may use a virtual machine (Ubuntu). And after download, you have to run the **update, upgrade** and **ssh script** on the **Your terminal.**
- And run the **ssh-keygen** on the terminal, just press enter on all input places. Your **key** will be saved on **/home/user/.ssh.** And goto **VM** see the **ip address of machine** by using **ifconfig**
- Run this command **ssh-copy-id -i id_rsa.pub VM(id)@VM(ip) .** And just **press enter** and write **yes** and **enter the pass of VM** “Example: **ssh-copy-id_rsa.pub deeocde@192.168.0.1**”
- Goto your terminal and enter this script.
- **#!/bin/bash**
- **FILE=”Backup-`date +%d-%m-%y-%H:%M:%S`.tar”**
- **tar -cvf /home/user/desktop/bash/$FILE /home/user/desktop/bash/**
- **scp /home/user/desktop/bash/$FILE VM(id)@VM(ip):/home/deecodes/**
- Save and run by crontab.
- **Save this script on crontab. like : * * * * * bash /home/deecode/desktop/bash/file.sh**
- It will be execute in every minute.

- **Server load monitoring script :-**

- Run this command first **grep pro /proc/cpuinfo -c**
- And run **cat /proc/loadavg**
- Create a script**.**
- **#!/bin/bash**
- **trigger=1.00**
- **load=`cat /proc/loadavg | awk ‘{print $1}’`**
- **response=`echo | awk -v T=$trigger L=$LOAD ‘BEGIN{if (L > T){print “Greater”}’`**
- **if [[$response = “Greater”]]**
- **then**
- **sar -q | mail -s “High server load - [$load]” user@gmail.com**
- And save the file and create a crontab. *** 1 * * * bash /home/deecode/file.sh**

51. **Project for penetration testing :-**
    - **XSS finder :-**
    - **#!/bin/bash**
    - **gau domain.com | grep “=” | qsreplace hack\” -a |**
    - #download the gau and qsreplace tool in your system.
    - **while read url**
    - **do**
    - **target=`curl -s -l $url | egrep -o ‘(hack”|hack\\”)’`**

- **echo -e “Target: \e[1;31m $url \e[0m” “$target” “\n”;**
- **done | sed ‘s/hack%22/[xss possible]Reflection found…./g’**
- Save and run the script. It will not be 100%. But ya try.

- **SSRF Finder :-**
- **#!/bin/bash**
- **banner () {**
- **clear**
- **echo**
- **echo -e “▛▘▛▘▛▀▖▛▘▜▘▛▀▖▛▀▖▌▒”**
- **echo -e “▙▖▙▖▌▞▛▖▌▒ ▒▌▒ ▙▄▞▙▄▞▙▄”**
- **echo -e “==========”**
- **echo -e “Usage: $0 -d, --domain [DOMAIN…] -s, --server [SERVER…]”**
- **}**
- **##**
- **if [$# == 0]**
- **then**
- **banner**
- **exit 1**
- **fi**
- **while [$# -gt 0]**
- **do**
- **case “$1” in**

- **“-h” | “--help”)**
- **banner**
- **exit 2**
- **;;**
- **“-d” | “--domain”)**
- **DOMAIN=$2**
- **shift**
- **shift**
- **“-s” | “--server”)**
- **SERVER=$2**
- **shift**
- **shift**
- **;;**
- ***)**
- **banner**
- **;;**
- **esac**
- **done**
- **if ! [-d $DOMAIN]**
- **then**
- **mkdir $DOMAIN**
- **cd $DOMAIN**
- **else**
- **echo “Directory already exists…exiting…”**
- **echo -n “[+] Do you want to delete the directory [y/n]:”**
- **read input**

- if [“input” == “y/Y”]
- then
- rm -rf $DOMAIN
- fi
- fi
- ##
- banner
- echo
- echo -e “Gathering URLS and replacing it with server URL…”
- gau $DOMAIN | grep “=” | qsreplace $SERVER | tee parameters.txt
- echo
- echo -e “[+]Completed…”
- echo
- echo -e “Firing urls check your server”
- cat parameters.txt |
- while read line
- do
- curl $line
- done
- echo
- echo -e “Completed…”
- echo -e “==============================”
- echo “Check Your server for any request…..”
- echo -e “==============================”

- Save the script and run. Before running the script. You have to configure the **ngrok**, you may get help from youtube. It just needs to see the request of the domain. To see ssrf attack or not. **Copy the Forwarding address from ngrok to paste on SERVER in the tool.** You may use burp collaborators.

- **URL extractor from javascript files :-**
- Here we have to create a file.
- **#!/bin/bash**
- **RED=”\e[31m”**
- **GREEN=”\e[32m”**
- **EXIT=”\e[0m”**
- **#----------**
- **echo**
- **echo -n -e “${RED}[+] Enter the name of directory to be created:${EXIT}”**
- **read dir**
- **mkdir $dir**
- **cd $dir**
- **echo**
- **echo -e “${RED}[+]JS files gathering…${EXIT}”**
- **cat $1 | grep -E -v “(*[.]json|*[.]jsp)” | grep -E *[.]js | tee js-extension-urls.txt**
- **echo**

- **echo -e “${GREEN}Gathering completed${EXIT}”**
- **echo**
- **echo -e “${RED}Sending HTTP requests…${EXIT}”**
- **cat js-extension-urls.txt |**
- **while read line**
- **do**
- **echo curl -I $line | grep -Eo “(http|https)://[a-zA-Z0-9./?=_%:-]*” | tee all_fecthed_urls.txt**
- **done**
- **echo -e “${GREEN}HTTP requests sent…${EXIT}”**
- Save the script and run. In this script the main is **grep or regex**.

52. **Bonus Script :-**
    - Here we learn about encrypting our code and they can run but they can’t see the code.
    - **sudo apt install shc**
    - **shc -f file.sh**
    - It will encrypt the file.

– Thank you –

written by
B.deepak choudhary